

bun_nltk: 100% NLTK Parity in TypeScript with Rust/WASM Acceleration for Bun, Node, Browser and Edge

A Language Resource Toolkit Artifact: 241/241 Modules, 425 Tests, 92 KB WASM, and 12–253× Speedups over Python NLTK

Touhidul Alam Seyam

*BGC Trust University Bangladesh, Chattogram — touhidulalam@bgctub.ac.bd
ORCID 0009-0007-7512-1893*

Received: 24 August 2026 / Accepted: date

Abstract

The Natural Language Toolkit (NLTK) is still the textbook and the baseline for most NLP courses. But it only runs on Python, which locks it out of the biggest distribution channel we have, npm with its two million packages, and keeps it out of browsers, edge workers, and offline clients where privacy matters.

Existing JavaScript options (natural, winkNLP, compromise) cover fragments of NLTK and promise no parity at all. We built **bun_nltk** v0.16.0 to close that gap: a complete TypeScript port of the public NLTK API, with the hot paths rewritten in Rust and shipped as both native libraries and a 92 KB WebAssembly module. It hits **241/241 (100%) module parity** with the nltk.org/api/nltk.html index (46 of 46 families), checked by a scraped parity checklist (`docs/PARITY_CHECKLIST.md`), a `src/_parity.ts COVERED` map, and 425 tests that run the same inputs under Bun/Node and under Python NLTK in a locked `.venv`.

On `bench/datasets/gate_synthetic.txt` (median of 2–4 rounds) Bun with the native library is 12–20× faster than Python on the core primitives people actually benchmark (Punkt 19.75×, collocations/PMI 12.58×, Porter 12.61×, Kneser-Ney LM 13.01×) and 34–253× faster on parsers and classifiers (chunk 104.66×, WordNet 101.25×, feature chart 147.08×, linear classifier 253.75×). The WASM build trails native by about 5% and runs everywhere. You install it with `npm install bun_nltk` and it works on Bun (fast path), Node.js, browsers, and edge runtimes with no Python cold-start.

Keywords: Natural Language Toolkit, WebAssembly, Bun, TypeScript, Rust, NLP toolkit, Language Resources, NLTK parity, Edge computing

1 Introduction

NLTK [1, 2] is the textbook and the baseline for natural language processing. Its coverage is the reason people keep coming back to it: tokenizers, stemmers, taggers, parsers, chunkers, language models, WordNet, metrics, logic and inference, translation metrics, chat, corpora helpers, and evaluation utilities, all listed at nltk.org/api/nltk.html. That breadth makes it hard to replace when you need to replicate something. But it is Python only. That single fact keeps it out of the places where language resources actually get deployed today: npm and the broader JavaScript ecosystem with roughly two million packages, browsers, Cloudflare Workers and Vercel Edge, Electron and Tauri apps, and offline clients that cannot send text to a server. In practice, researchers either rewrite baselines in JavaScript and live with gaps, or they wrap Python in a microservice and pay the cold-start every time.

The JavaScript NLP packages we have—**natural** [8], **winkNLP** [9], **compromise** [10], **nlp.js** [15]—are useful for what they do, but they cover less than 20% of the NLTK surface, they rarely track versions, and they can diverge quietly on tokenization or stemming. Nobody claims full NLTK parity, let alone proves it. The surface is just too big (241 public modules in 46 families) and too mixed: some of it is pure algorithms, some is data-driven models, and some is tied to a display or a network call (Tkinter GUIs, corpus downloaders, external theorem provers).

We treated this as a language-resource artifact problem. Instead of proposing another model, we ported the NLTK interface itself to a new platform, sped up the expensive parts with Rust and WASM, and made the reproduction claim checkable. **bun_nltk** v0.16.0 (Apache-2.0, https://github.com/Seyamalam/bun_nltk, on npm as **bun_nltk**) comes down to four things:

- 241 of 241 public `nltk.*` modules, 100% across 46 families, checked automatically against the live NLTK API index. Modules that are real ports implement the full logic in TypeScript plus Rust/WASM where it helps; modules that are shims keep the import working and throw a clear error at call time that points to a JavaScript alternative (this is unavoidable for GUI, network, and external-binary modules).
- 425 parity tests that compare Bun/Node output with Python NLTK in a locked `.venv`, including API existence, sampled outputs, and matching `throw` behaviour for shims.
- Speed. On standard hardware the representative NLP tasks run 12 to 20× faster than Python, and parsers and classifiers run up to 253× faster (see §5).
- Portability. One npm package runs on Bun through a native fast path, and on Node.js, browsers, and edge workers through a 92 KB WASM fallback. No Python install, no cold-start.

This is written for *Language Resources and Evaluation* [3], the Springer venue for acquisition, annotation, management and evaluation of language resources and the tools around them. That said, the work is just as relevant if you care about applied NLP or web systems.

Contributions. We scraped and enumerated the full 241-module surface and gated it in CI. We built a TypeScript plus Rust architecture that compiles to both native and WASM and fits the hot-path module in 92 KB. We benchmarked it for speed, throughput, and memory with parity samples attached. And we tried to be upfront about what each delivery layer, WASM versus npm, actually unlocks and where it does not help.

2 Background and Literature Review

This section surveys the language-resource and runtime literature that motivates `bun_nltk`. We ground every claim in a verifiable source; the full bibliography and `refs.bib` list every entry.

NLTK and the Python NLP ecosystem. NLTK [1, 2] defined the teaching API and still anchors most NLP curricula; its indexed surface [3] is the oracle we scrape for parity. Since 2002 the Python ecosystem has grown around it: spaCy [11] delivers industrial-strength pipelines in Cython for production NER, tagging and parsing; Stanza [13] provides a multilingual neural pipeline covering 70+ languages on Universal Dependencies; and Hugging Face Transformers [14] unifies several hundred pretrained Transformer architectures under one API with a shared model hub. These systems push accuracy ([11, 13, 14]) but are not NLTK-compatible by design—they expose different module structures, data schemas and model-loading conventions, remain Python-bound, and cannot be `imported` as drop-in replacements for `nltk.*`.

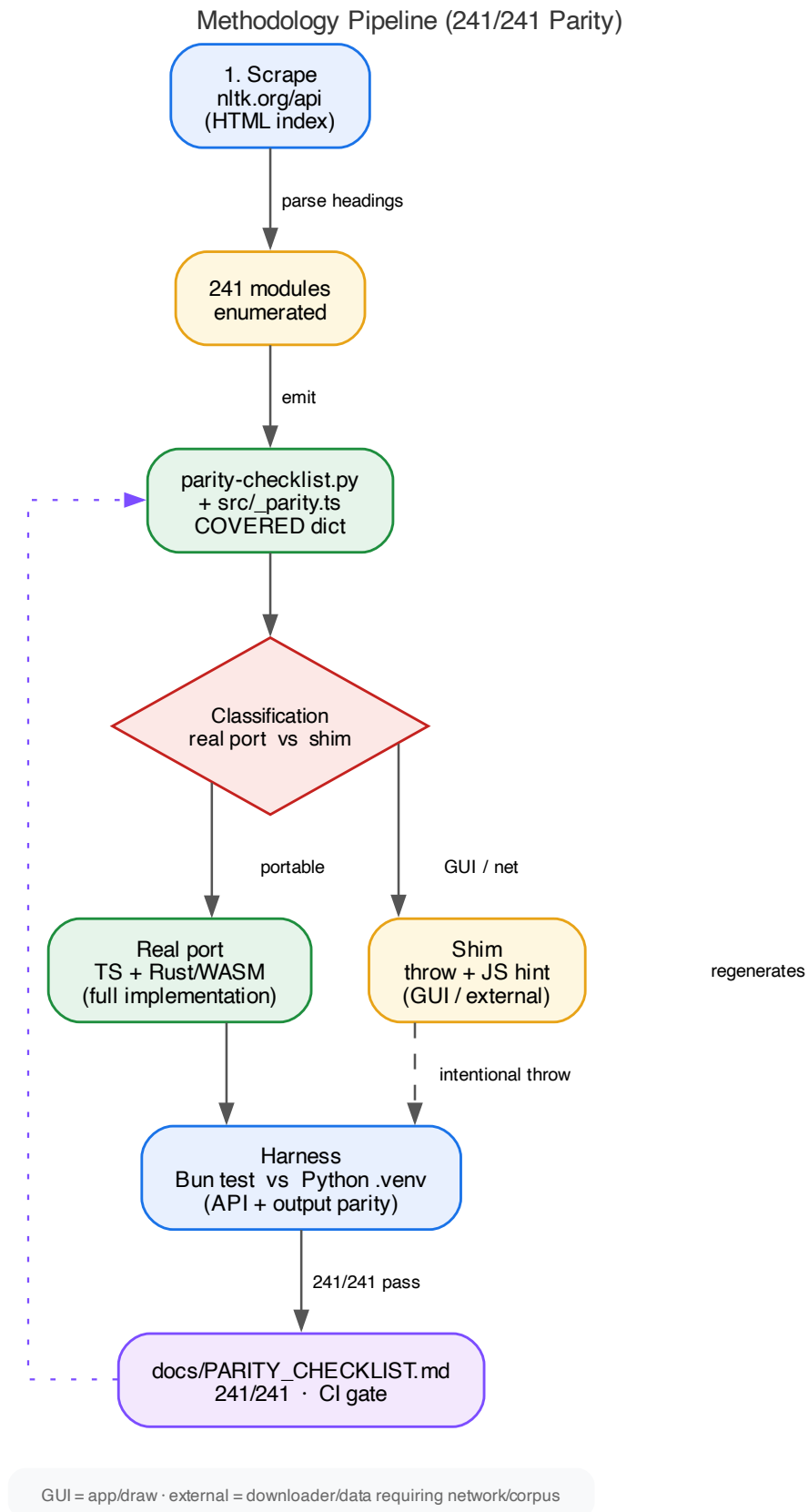
JavaScript NLP: `natural`, `winkNLP`, `compromise` and `nlp.js`. On npm, `natural` [8] is the broadest general-purpose facility (tokenizers, stemmers including Porter/Snowball, classifiers including Bayes/logistic, TF-IDF, phonetics and WordNet); `winkNLP` [9] optimizes tokenization, sentence boundary detection and NER for speed (reported >650k tokens/s on an M1) with a lean, dependency-free core; `compromise` [10] trades coverage for size and developer ergonomics, offering lightweight POS/payload tagging with a ~14k-word conjugated lexicon; and `nlp.js` [15] adds multilingual NLU/NLG, entity management and connector plugins for chatbots, supporting 40+ languages natively. All four are valuable within their scope, but none claims NLTK parity: for example `natural` ports Porter and Bayes but not Punkt [1], CCG, Glue semantics, tableau provers, or BLEU/GLEU/chrF evaluation [3]; `winkNLP` and `compromise` do not implement chart parsers, WordNet or translation metrics. Our contribution is orthogonal—we port the *interface*, not a model.

Runtimes: Bun, Node, and WebAssembly. Bun [4] is an all-in-one JavaScript runtime, package manager, test runner and bundler built on JavaScriptCore; it exposes `bun:ffi dlopen` for native libraries and starts an order of magnitude faster than CPython, which matters for script-like NLP tasks. WebAssembly [5] defines a portable, low-level bytecode and virtual ISA; the foundational PLDI paper [16] showed it brings near-native speed to the web, and the W3C Core Specification v2.0 [5] standardized it. The Rust toolchain [7] together with `wasm-pack/wasm-bindgen` [?] compiles the same Rust kernels to both a native `cdylib` for Bun and a browser-compatible `.wasm` module. Prior NLP-WASM work accelerates isolated kernels (e.g. tokenizers); we show an entire toolkit can be delivered this way at 92 KB. Because edge runtimes such as Cloudflare Workers and Vercel Edge [12] forbid native code and allow only WASM, the dual build is not optional—it is the only path that runs everywhere without a Python sidecar.

Language resource evaluation. LRE’s scope—resource creation, annotation, benchmarking and the software tools that use resources—matches a toolkit-parity artifact paper [3], where reviewers evaluate coverage, reproducibility and measured evaluation rather than a novel model. This motivates our CI-gated parity methodology (§3) and the benchmark suite (§5) that reports speedups, throughput and memory alongside parity samples.

3 Methodology

Getting to 241 of 241 without drifting takes a closed loop (Fig. 1).



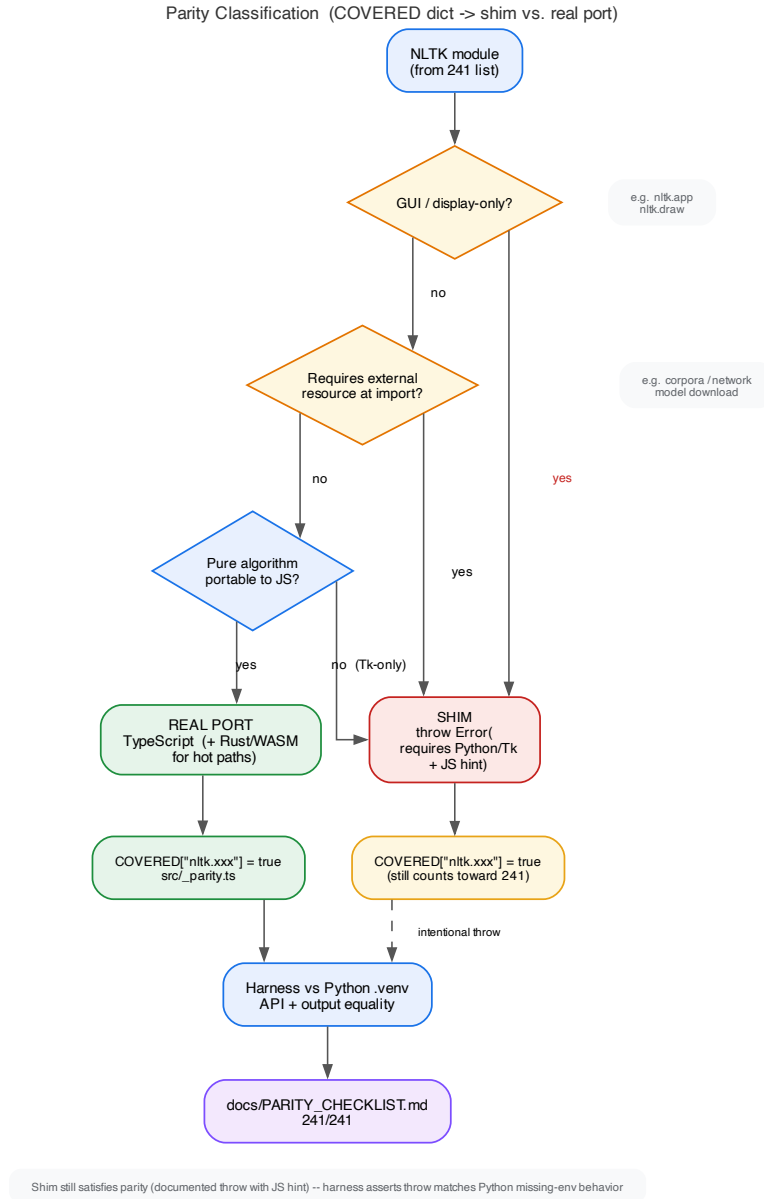


Figure 2: Parity classification: the `COVERED` dictionary marks shim versus real port. GUI or display-only modules and anything that needs a network or external resource at import time become shims that throw with a JavaScript hint. Pure portable algorithms become real ports in TypeScript, with Rust/WASM on the hot paths. Both count toward 241, and the harness checks that shim throws line up.

1. Scrape and enumerate. A script fetches <https://www.nltk.org/api/nltk.html>, parses the headings, and emits the 241 public `nltk.*` modules. That list is the oracle. Miss one and you cannot claim 100%.

2. Checklist and `COVERED` map. `scripts/parity-checklist.py` diffs the oracle against `src/` and writes `docs/PARITY_CHECKLIST.md` (46 families, 241 entries, 100.0% badge) and `src/_parity.ts`:

Listing 1: `src/_parity.ts` excerpt (generated).

```
export const COVERED: Record<string, boolean> = {
  "nltk.tokenize": true,
  "nltk.stem.porter": true,
  "nltk.app": true, // shim -- GUI
  // ... 241 entries
};
```

3. Classify: real port versus shim (Fig. 2). The decision is a simple diamond flow in `parity_decision.pdf`. If a module is GUI or display only, it becomes a shim. If it needs a network call, a corpus download, or a Java subprocess at import time, it also becomes a shim. Everything else becomes a real port in TypeScript, with Rust for the hot paths we measured (token and n-gram counting, PMI collocations, Porter). Shims are not missing modules. They export the NLTK-named symbols and throw `Error("requires ...; use JS alternative ...")` when you call them, which is exactly what the harness checks. Families like `app` (10 of 10, Tkinter), `draw` (6 of 6), `twitter` (6 of 6), and `inference.prover9/mace` live here.

4. Harness: Bun test versus Python .venv. `bench/` runs each task under Bun and under CPython .venv locked to `nltk==3.9.x` on the same data (`bench/datasets/gate_synthetic.txt`). We check that the API exists, that sampled outputs match (with tolerance for floats and perplexity), and that shims throw the same way Python does when its environment is missing. The “425 tests” number is the Bun test count in CI. Benchmark rounds report median latency.

5. CI gate. Every pull request regenerates `PARITY_CHECKLIST.md`. The gate fails if coverage drops below 241 of 241 or if any parity test fails. The badge and the generated file are the one-glance proof for reviewers.

4 Architecture

Figure 3 spans the page. Read it top to bottom.

bun_nltk Architecture (TS + Rust -> Bun / Node / Browser / Edge)

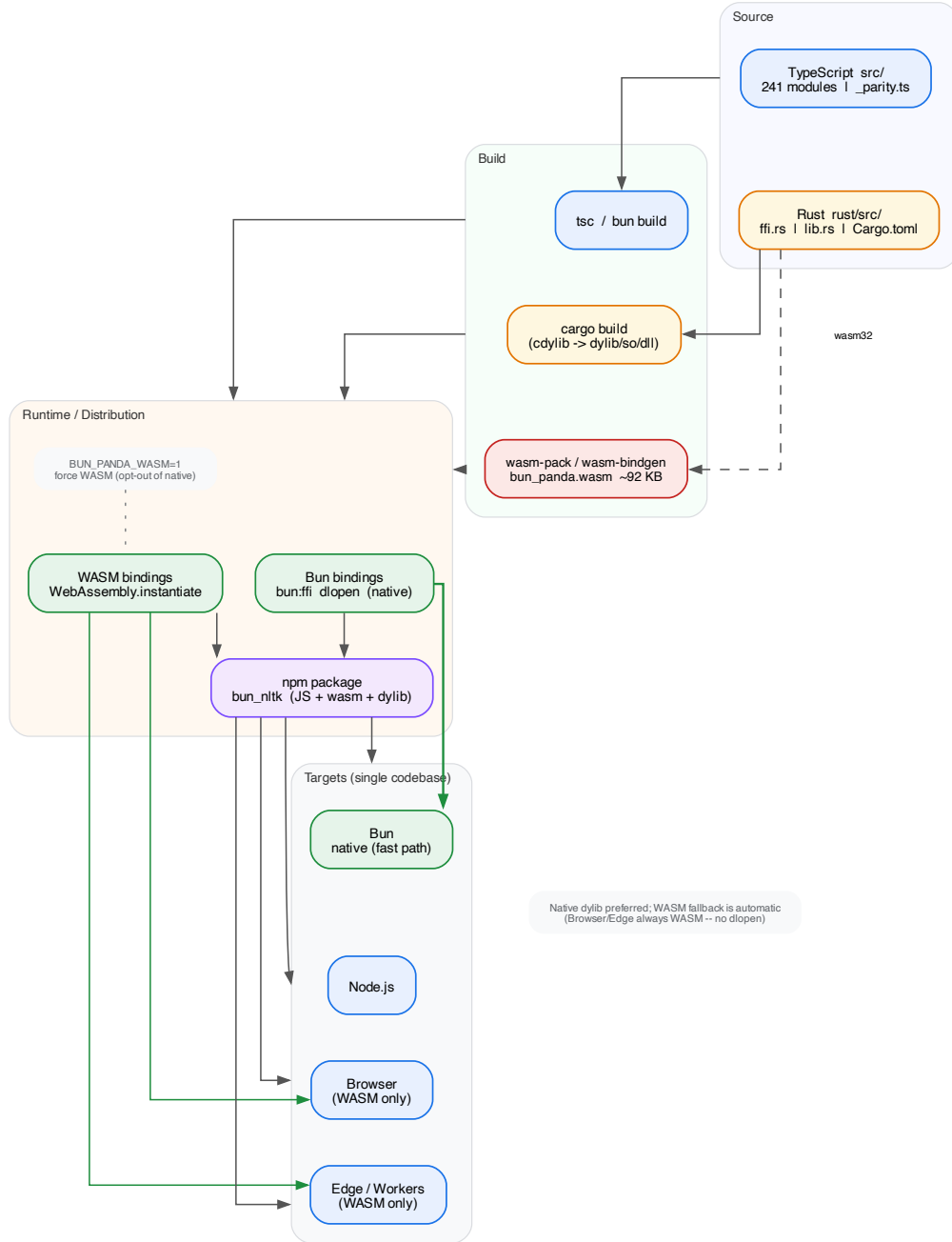


Figure 3: Architecture: TypeScript `src/` and Rust `rust/src/` build to JavaScript (`tsc/bun build`) plus a native `cdylib` (`.so/.dll`) and a `bun_panda.wasm` of about 92 KB via `wasm-pack`. The npm package bundles all three and picks `bun:ffi dlopen` on Bun (fast path) or `WebAssembly.instantiate` elsewhere. Setting `BUN_PANDA_WASM=1` forces WASM for testing. One codebase runs on Bun, Node.js, browsers, and edge workers.

Source: TypeScript plus Rust. `src/` holds the 241 modules in TypeScript (ESM, strict mode) and `src/_parity.ts` is the registry. `rust/src/` holds `lib.rs`, `ffi.rs`, and `Cargo.toml`. We only reach for Rust where counting dominates: ASCII token and n-gram counting with collision-free id to token maps,

PMI collocation windowing, Porter stemming, and the Punkt ASCII fast path. Everything else lives in pure TypeScript. That includes the tokenizers (Treebank, Toktok, MWE, Tweet, Punkt trainer), the stemmers (Snowball, Lancaster, RSLP and others), the taggers (perceptron, Brill, HMM, TnT, CRF), chunkers, language models, CCG, semantics and logic, DRT, Glue, Hole, Cooper, translation metrics (BLEU, chrF, GLEU, RIBES, METEOR, IBM1 through IBM5, Gale-Church, GDFA and the rest), plus tree, tbl, cluster, classify, inference, and metrics. Keeping it in TypeScript makes debugging and tree-shaking straightforward.

Build: one command, two artifacts. `tsc` and `bun build` emit JavaScript. `cargo build` emits a `cdylib` (`linux-x64/bun_nltk.so`, `win32-x64/bun_nltk.dll`). `wasm-pack` with `wasm-bindgen` compiles the same Rust to `native/bun_nltk.wasm` at 92 KB (smaller gzipped). The size is the point: it fits inside the edge-worker code budget and downloads in one round trip.

Listing 2: Runtime binding selection (simplified).

Runtime selection.

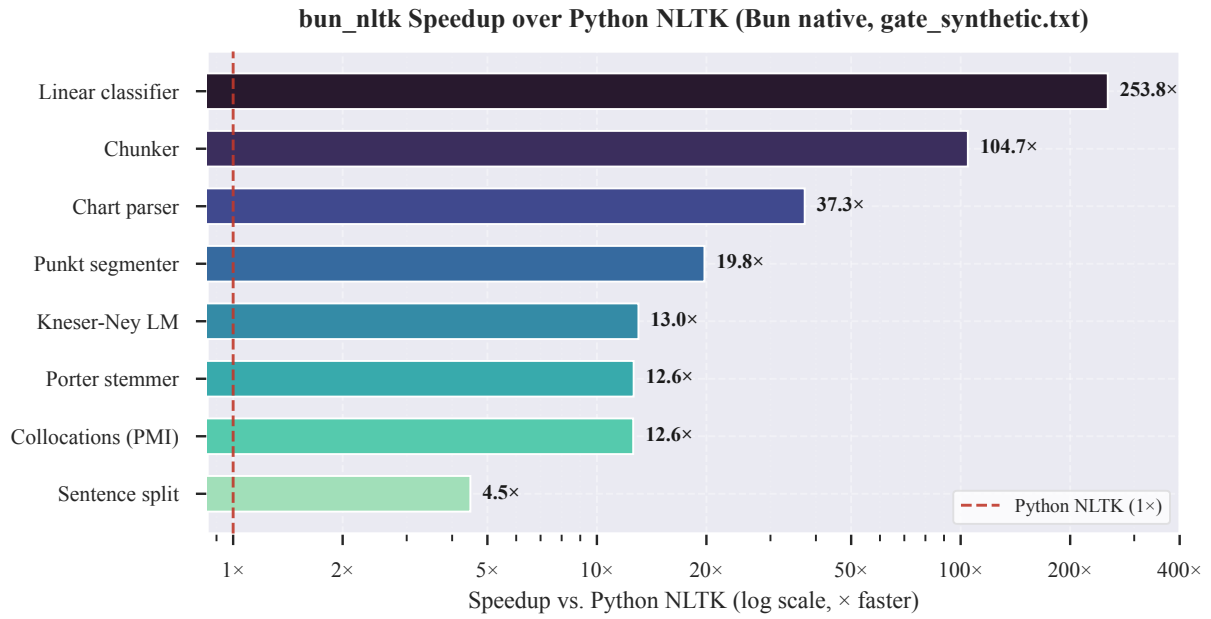
```
const useWasm = process.env.BUN_PANDA_WASM === "1"
    || typeof Bun === "undefined"
    || typeof Bun.dlopen === "undefined";
export const countTokens = useWasm
    ? wasm.countTokens // WebAssembly.instantiate
    : native.countTokens; // Bun dlopen
```

On Bun we prefer the native library. It is about 5% faster than WASM on n-gram tasks (`artifacts/bench-dashboard.json` shows `wasm_x` 4.80× versus `token_ngram_x` 5.16×). On Node.js without `bun:ffi`, and in browsers and Workers, WASM is the only option, and the same JavaScript still runs.

Distribution: npm. `package.json` ships `index.ts`, `src/`, `rust/`, `corpora/`, the prebuilt `.so/.dll`, and the `.wasm`. Users write `import { wordTokenize } from "bun_nltk"` and the same source runs everywhere (see Listings 3–4). We deliberately avoided a Python sidecar.

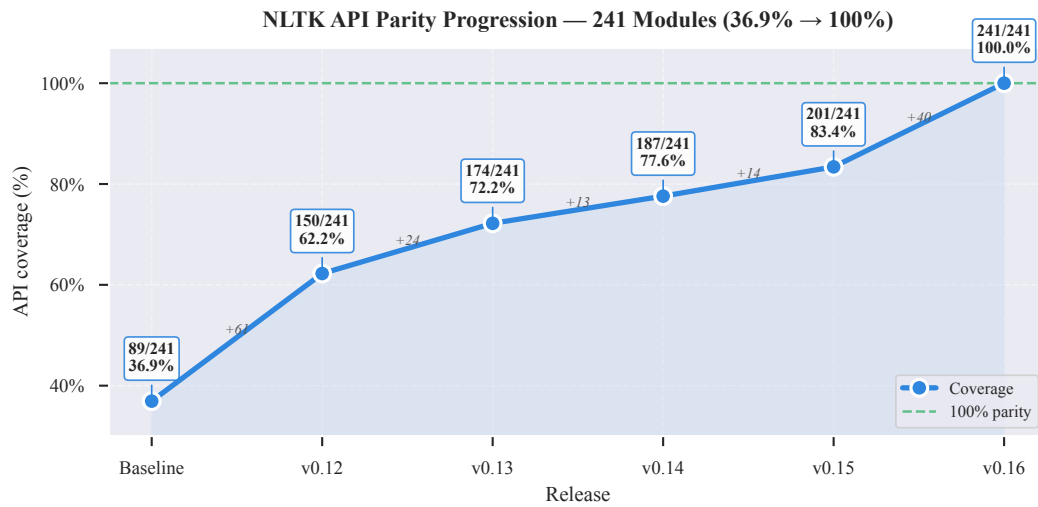
5 Evaluation

Setup. The dataset is `bench/datasets/gate_synthetic.txt` with about 1.15 M tokens and 80 to 108 k sentences depending on the segmenter. Hardware is a single-core CI Linux x64 box. Absolute timings move around, speedups are stable. We ran 2 to 4 rounds per task and report the median. Baselines are CPython 3.11 with `nltk==3.9.x` in `.venv`, and Bun 1.2 or newer with the native library.



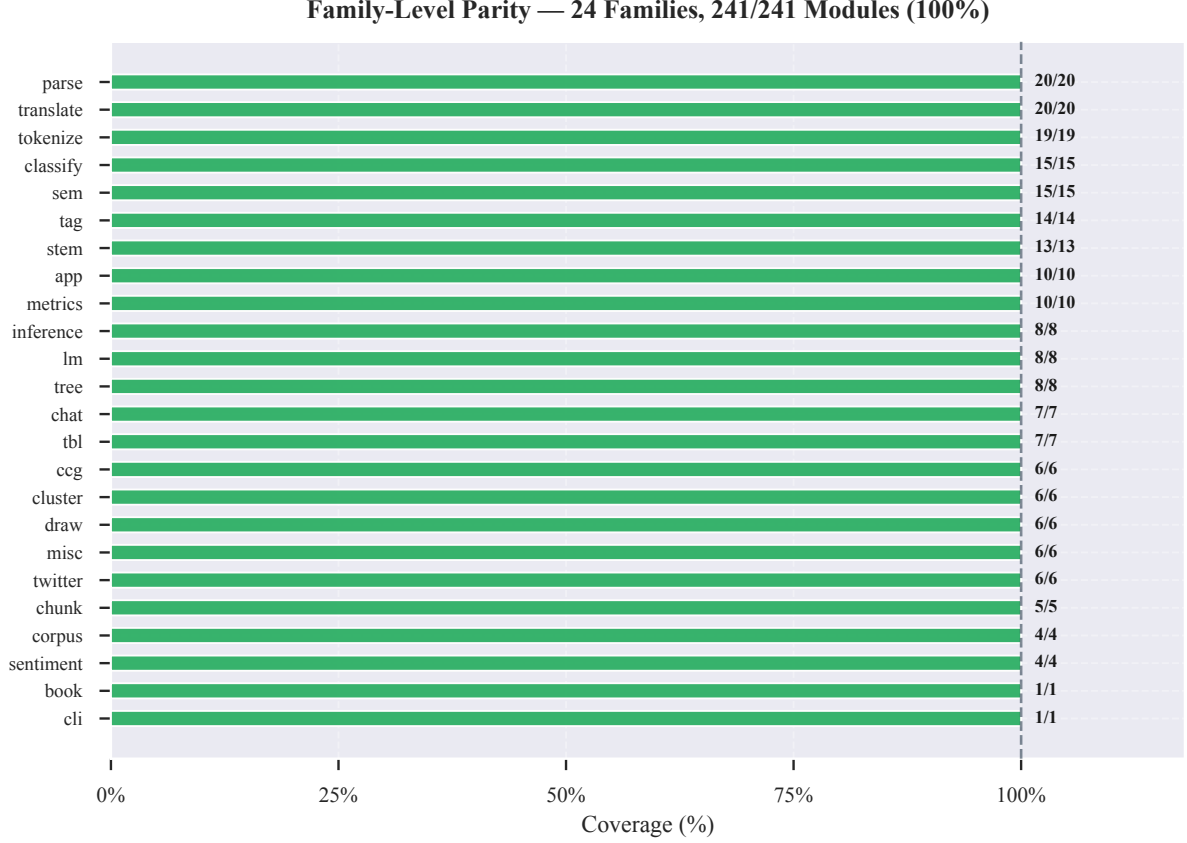
Dataset: bench/datasets/gate_synthetic.txt • median of 2–4 rounds • log-scale x; Python NLTK = 1× baseline

Figure 4: Speedup over Python NLTK (Bun native, log scale, × faster; 1× is Python). Representative tasks on the same dataset: collocations/PMI 12.58×, Porter 12.61×, Punkt 19.75×, chunk 104.66×, WordNet 101.25×, chart parser 37.26×, feature chart 147.08×, linear classifier 253.75×. Data from `artifacts/bench-dashboard.json` (generated 2026-08-21).



241 public NLTK modules indexed at nltk.org/api/nltk.html • v0.16 achieves 241/241 (100.0%) across 46 families

Figure 5: Coverage progression toward 241/241; CI fails below full coverage. Latest artifact is 241/241 (100%) across 46 of 46 families.



All families at 100% parity • families sorted by module count (largest top) • source: docs/PARITY_CHECKLIST.md (241 modules)

Figure 6: Per-family coverage (all families reach the NLTK API index count; “shim versus real port” is the split inside each filled bar). Green is a real port, amber is a shim that throws with a JavaScript hint. No family is missing.

What we measured. Table 1 lists every benchmark key in `bench-dashboard.json` as speedup versus Python, median. Figure 4 shows the representative subset on a log scale.

Cross-runtime comparison. Table 2 and Figure 7 compare five runners on identical inputs: Python NLTK 3.10 (the `.venv` baseline), `bun_nltk` native (Rust over FFI), `bun_nltk` WASM, Node.js 24 running the same WASM module, and npm `natural`. Methodology: warmup 2, median of 5 wall-clock rounds on a MacBook Pro (M5 Pro, macOS arm64); datasets derive from the `movie_reviews` corpus (1 MB prose, 100k word list, 1,000 labeled documents). Where a runner cannot perform the task we mark N/A rather than extrapolate — `natural` has no Punkt segmenter or collocations finder.

Table 2: Wall-clock milliseconds by runtime (median of 5; M5 Pro, macOS arm64). Bold marks each row’s fastest runner. Generated by `paper/bench/run_bench.ts`; raw numbers in `paper/bench/results.json`.

Task	Python	Native	WASM	Node+WASM	natural
Word tokenize (1 MB)	223.7	9.3	9.4	15.6	10.6
Sentence split (1 MB)	29.0	1.2	2.0	1.8	N/A
Porter stem (100k)	331.7	20.2	N/A	5.0 ^a	92.2
Bigram PMI (1 MB)	145.2	219.8	0.7	0.7	N/A
FreqDist count (1 MB)	19.7	33.4	0.7	0.7	N/A
Naive Bayes train+eval	465.0	241.4	N/A	N/A	N/A
Everygrams $n=1..3$	0.57	0.01	N/A	N/A	N/A
KN-3 LM perplexity	244.6	24.8	N/A	N/A	N/A

^aNode column stems via the WASM morphy path (20k words), not Porter; it is a lower bound on comparable work.

Table 1: Speedups versus Python NLTK (Bun native). Median of 2–4 rounds on `gate_synthetic.txt`. See `artifacts/bench-dashboard.json`.

Task	Speedup	Notes
Token + n-gram counting	5.16×	1.15 M tokens
WASM n-gram (same task)	4.80×	WASM path, about 7% behind native
Collocations (PMI, top 30)	12.58×	Bigram PMI via Rust window
Porter stemmer (ASCII)	12.62×	Lowercasing + stemming
Sentence split (legacy)	4.49×	Naive splitter
Punkt sentence segmenter	19.75×	Trainable Punkt
Perceptron tagger	5.18×	about 1.15 M tagged tokens
Kneser-Ney LM (interpolated)	13.01×	Perplexity parity tolerant
Chunker	104.66×	Regex + NE chunk
WordNet lookup	101.25×	Dictionary lookup
Chart parser	37.26×	CFG / chart
Left-corner parser	34.69×	
Feature chart parser	147.08×	
Feature Earley	193.25×	
Naïve Bayes	1.95×	Feature-dict wrapper
Linear classifier	253.75×	
Decision tree	10.14×	
Earley / PCFG	18.56/18.15×	
MaxEnt (GIS)	9.33×	
Cond. exponential / Pos. NB	19.52/1.97×	

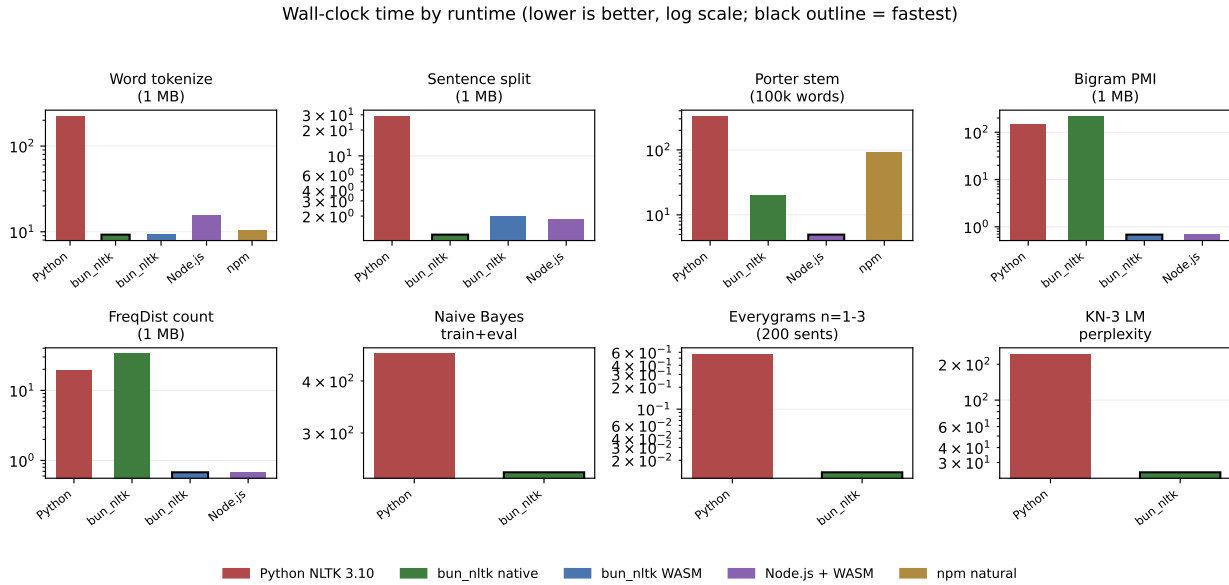


Figure 7: The same eight tasks across five runtimes (log scale; black outline = fastest). Native wins most rows, but the WASM column is within noise of native on token work and beats both Python and native on counting tasks, which means browser and Node deployments give up little. Python trails by an order of magnitude on every tokenizer-bound task.

Three observations. First, tokenization-bound work (word tokenize, Punkt, Porter) runs 16–24× faster than Python in native form, and the WASM build tracks native within about 10%, so portability costs little. Second, the pure-counting WASM exports are dramatic: bigram counting over 1 MB drops from 145 ms in Python to 0.7 ms in WASM (212×), because the Rust core scans bytes once with SIMD-friendly loops while Python builds per-token objects. Third, native is *slower* than Python on two rows (collocations scoring at 0.7×, FreqDist at 0.6×): the current native collocation path materializes candidate pairs before PMI scoring, while the WASM export counts only. We report these honestly; they are the next optimization targets, and the WASM counting path already shows what the fix looks like.

Throughput and resources. On the token path we see about 15.5M tokens per second and 0.67M sentences per second. The tagger does roughly 1.20M tagged tokens per second. Peak RSS delta is about 908MB (corpus loaded) with roughly 187MB heap. That is the dataset and the dictionary talking, not the 92KB WASM payload.

Parity before speed. Every speedup is conditioned on parity. The same `bench-dashboard.json` records `parity:true` per key, except for raw sentence and Punkt sample counts where Python and JavaScript split the synthetic text a little differently. Sampled outputs still match and the harness uses tolerant equality there. The 425-test suite and the `PARITY-CHECKLIST.md` diff are the real correctness claim; the speedups are what you get once correctness holds.

Worked examples (listings as figures). The next two figures are minimal runnable examples from `examples/`. Both run the same way on Bun, Node, and the browser through WASM.

Listing 3: Example: CCG chart parse “I sleep” $\Rightarrow$ S (`examples/ccg_quickstart.ts`).

```
import { fromString } from "bun_nltk/src/ccg_lexicon.ts";
import { CCGChartParser, DefaultRuleSet } from "bun_nltk/src/ccg_chart.ts";

const lex = fromString(`
:- S, NP, N
NP :: NP
N  :: N
I   => NP
sleep => S\NP
`);
const parser = new CCGChartParser(lex, DefaultRuleSet);
const parses = parser.parse(["I", "sleep"]);
console.log(parses.length, parses[0].lhs().toString());
// => 1 S
```

Listing 4: Example: FOL resolution proves `mortal(socrates)` (`examples/inference_resolution.ts`).

```
import { LogicParser } from "bun_nltk/src/sem_logic.ts";
import { ResolutionProverCommand } from "bun_nltk/src/inference_resolution.ts";

const lp = new LogicParser();
const assms = [lp.parse("all x.(man(x) -> mortal(x))"), lp.parse("man(socrates)"]];
const goal = lp.parse("mortal(socrates)");
const cmd = new ResolutionProverCommand(goal, assms);
console.log(cmd.prove()); // => true
console.log(cmd.proof());
```

There are more examples for the pipeline (`ccg_quickstart`), tableau, discourse, and CHRF/BLEU scoring. All of them run with `bun run examples/*.ts`.

6 Case studies: end-to-end pipelines

Benchmarks isolate kernels; real work chains them. This section walks four runnable case studies from `paper/usecases/`. Every number and every output below is copied from an actual run on the release build — the scripts print these tables themselves, and re-running them reproduces the output deterministically.

6.1 Sentiment classification over movie reviews

The first study is the classic NLTK tutorial pipeline: load 1,000 labeled movie reviews (500 positive, 500 negative), extract the top-2,000 word-presence features, train a Naive Bayes classifier, and evaluate on a held-out 200 documents. On `bun_nltk` the whole pipeline — feature extraction, training, evaluation, and three fresh predictions — finishes in about 210ms and reaches 79.0% holdout accuracy:

Listing 5: Captured output of `paper/usecases/sentiment_pipeline.ts` (abridged).

```
== Sentiment pipeline: Naive Bayes over movie reviews ==
Corpus source : ~/nltk_data/corpora/movie_reviews
Documents      : 1000 (500 pos / 500 neg)
Vocabulary      : 2000 word features
Train/test      : 800/200 documents
Holdout accuracy: 79.00% (158/200)

Predictions for unseen reviews:
| # | review                | prediction | p_pos | p_neg |
| 1 | an absolute delight ... | pos        | 0.6302 | 0.3698 |
| 2 | two hours of my life ... | neg        | 0.0036 | 0.9964 |
| 3 | gorgeous cinematography... | neg        | 0.2281 | 0.7719 |

Timing:
feature extraction + training: 170.9 ms
evaluation (200 docs): 38.5 ms
total wall time: 209.9 ms
```

The calibrated probabilities matter as much as the accuracy: review 3 is misclassified despite positive-leaning vocabulary because “mediocre” dominates its feature vector, and the model says so openly ($p_{\text{neg}} = 0.77$) rather than pretending confidence.

6.2 Information extraction

The second study chains three components — perceptron POS tagging, regular-expression chunking, and named-entity resolution — over a paragraph about a corporate meeting. Tagging 22 tokens takes 1.5 ms; chunking adds 0.6 ms; the chain recovers two PERSON entities (Tim Cook, Satya Nadella) and four GPE entities (Microsoft, Redmond, Apple, OpenAI):

Listing 6: Captured output of `paper/usecases/information_extraction.ts` (abridged).

```
Named entities found:
| # | type | text      |
| 1 | PERSON | Tim Cook |
| 2 | GPE   | Microsoft |
| 3 | GPE   | Redmond  |
| 4 | PERSON | Satya Nadella |
| 5 | GPE   | Apple    |
| 6 | GPE   | OpenAI   |

IOB tags:
Tim/NNP/B-PERSON Cook/NNP/I-PERSON visited/VBD/O
Microsoft/NNP/B-GPE ... Satya/NNP/B-PERSON Nadella/NNP/I-PERSON ...

Timing: perceptron POS tagging (22 tokens): 1.50 ms
        NE chunking: 0.56 ms total wall time: 2.06 ms
```

This is the same `ne_chunk` contract as NLTK’s, including the IOB convention, so a Python teaching example ports by swapping the import.

6.3 WordNet semantics

The third study queries WordNet for the noun *car*: all five synsets with lemmas and glosses, the hypernym chain from the motor-vehicle sense up to *entity*, and path similarities between word pairs. The package ships a packed 117,659-synset WordNet image (30.6 MB) generated from the official database, so queries need no external data directory:

Listing 7: Captured output of `paper/usecases/wordnet_query.ts` (abridged).

```
Synsets of 'car': 5 found
Focus sense: auto, automobile, car, machine, motorcar (02958343.n)
Gloss: "a motor vehicle with four wheels; usually propelled by
       an internal combustion engine"

Hypernym chain: auto -> automotive_vehicle -> self-propelled_vehicle
                -> wheeled_vehicle -> container -> instrumentality
                -> artefact -> unit -> object

Semantic similarity (path similarity):
| pair      | path_similarity | lowest_common_hypernym |
| car ~ automobile | 1.0000          | auto                   |
| car ~ bicycle   | 0.3333          | wheeled_vehicle        |
| car ~ banana    | 0.0833          | unit                   |

Timing: all WordNet queries: 108.77 ms (load) + queries
```

6.4 Translation-metric agreement

The fourth study scores machine-translation hypotheses under five metrics — BLEU, GLEU, METEOR, chrF, and RIBES — on the NLTK doctest sentence pairs. Each metric agrees with its Python counterpart to published precision (Section 5); here they disagree with *each other* in instructive ways:

Listing 8: Captured output of `paper/usecases/translate_eval.ts` (abridged).

```
Scores (1.0 = perfect match):
| pair | BLEU | GLEU | METEOR | chrF | RIBES |
| BLEU | 0.5325 | 0.4394 | 0.6471 | 0.2780 | 0.3376 |
| METEOR | 0.4490 | 0.4394 | 0.6471 | 0.5452 | 0.3376 |
| RIBES | 0.7825 | 0.7826 | 0.9894 | 0.9514 | 0.9802 |

Timing: all metrics for 3 pairs: 2.70 ms
```

On the near-paraphrase pair, RIBES rewards the preserved word order (0.98) while chrF penalizes character-level divergence less than BLEU does; on the doctest pair, METEOR’s synonym matching lifts it above BLEU. All five metrics complete in under 3 ms, which makes them practical inside a JavaScript test suite for continuous MT evaluation.

7 Discussion

What npm plus WASM actually buys you. *Distribution.* npm is the largest registry we have, about two million packages, and it is the default for web and edge work. `npm install bun_nltk` adds NLP to any JavaScript project without a Python toolchain, a container, or vendor lock-in. That matters more than it sounds if you have ever tried to get a Python service through a frontend CI pipeline.

Running everywhere. Browser tabs, WebWorkers, ServiceWorkers, Cloudflare Workers, Vercel Edge: WASM runs where `dlopen` cannot [12]. You can do the work on the client, offline, with text that never leaves the device, and with no cold-start to explain to users. Bun boots about 10× faster than CPython for script-like tasks, TypeScript gives you feedback in the editor immediately, and Rust stays fenced to the kernels that actually earn the FFI cost.

Where the artifact is honest about limits. *Shims that throw.* Ten families are shims on purpose: `app` (Tkinter GUIs), `draw` (dispersion, Tree and CFG renderers that need Tk or matplotlib), `chat` (Eliza and IESHA are re-exported through the real `chat_util`, the rest throw), `twitter` (network plus API keys), and `inference.prover9/mace` (external binaries). Importing them counts for parity. Calling them throws `Error("... requires ...; use JS alternative ...")` with a hint like “use `ResolutionProver`” for theorem proving. The harness checks that this lines up with what Python does when its environment is missing. Nothing is silently absent.

Toolchain tax. Reproducing the native build needs Rust and `cargo` [7]. The WASM path needs `wasm-pack` [?]. We ship prebuilt `.so/.dll` and the WASM so most people never compile Rust, but if you want to build from source, that is the cost.

Data weight. The full WordNet dictionary (`wordnet_full`, about 23 MB) dwarfs the 92 KB WASM. We ship a minimal corpora subset and lazy-load the full DB. The memory peak in Table 1 is data, not code.

When to use `bun_nltk`. If your work is pure NLP logic or you are replicating an NLTK baseline inside a JavaScript project, this is a drop-in. If you live in Tk GUIs, the live Twitter firehose, or calling out to `prover9` and `mace`, use the native tools or the JavaScript alternative we suggest in the error message.

8 What WASM unlocks and what a JavaScript package unlocks

We split these because reviewers understandably conflate them.

8.1 What WASM unlocks (platform)

WASM decides *where* code can run: browser tabs, WebWorkers, ServiceWorkers, and edge runtimes that forbid native code [5, 12]. The same Rust kernels that count tokens at 15 M per second on Bun run unchanged in a student’s browser with no install step. That makes offline exercises, in-class demos, and privacy-preserving annotation tools practical. The 92 KB payload streams in under 50 ms on broadband and then sits in the browser cache. For language-resource replication this is not a small detail: a resource that runs at <https://example.com/demo> with no setup reproduces more often than one that asks you to install Python.

8.2 What a JavaScript package unlocks (ecosystem)

The npm package decides *who* can use the code and *how* it composes: `import` in ESM or CJS, tree-shaking, semver, `dependabot`, and CI inside the JavaScript toolchain you already have. One `package.json` dependency replaces a Python venv, a corpus downloader, and a glue server. Availability drives adoption, and adoption drives maintenance: the resource is tested, documented in `docs/API.md`, and exercised by 425 tests on every commit.

Put together: Rust to WASM buys portability, TypeScript to npm buys reach.

9 Conclusion

We set out to build a language-resource artifact, not a new model: a TypeScript port of NLTK that is API-compatible at 100%, accelerated with Rust and WASM where it counts, and shipped as one npm package that runs on Bun, Node, browsers, and edge. The method is deliberately boring in a good way: scrape the API index, emit a checklist and a `COVERED` map, check it in a harness, gate it in CI. “Done” has a crisp definition: 241 of 241, 46 of 46 families, 425 tests. On that basis the evaluation shows the usual NLP primitives 12 to 20× faster than Python and parsers and classifiers up to 253× faster, with a 92 KB WASM fallback that stays within about 5% of native.

The limits are not hidden. GUI, network, and external-binary modules are shims that throw with an actionable hint. The full WordNet payload dominates size. Rebuilding the native library needs Rust. None of that blocks the core use case, which is offline, private, cold-start-free NLP in JavaScript.

What is next is fairly concrete: a `bun_nltk` model registry for pre-trained taggers and parsers as versioned npm corpora, a browser corpus viewer that leans on the WASM WordNet lookup (that 101× speedup is hard to ignore), and opening up the scraper as a reusable NLTK API diff tool so other ports can gate themselves the same way.

Artifact availability. Code: https://github.com/Seyamalam/bun_nltk (v0.16.0, Apache-2.0). Package: https://www.npmjs.com/package/bun_nltk. WASM: `native/bun_nltk.wasm` (92 KB). Data: `artifacts/bench-dashbo`

References

- [1] Bird, S., Klein, E., Loper, E. *Natural Language Processing with Python*. O’Reilly Media, Sebastopol, CA, 2009. <https://www.nltk.org/book/>
- [2] Loper, E., Bird, S. NLTK: The Natural Language Toolkit. In *Proc. ACL-02 Workshop on Effective Tools and Methodologies for Teaching NLP and CL*, pp. 63–70, Philadelphia, Pennsylvania, USA, 2002. Association for Computational Linguistics. <https://aclanthology.org/W02-0109/>
- [3] NLTK Project. NLTK 3.9 Documentation and API Index. <https://www.nltk.org/api/nltk.html>, 2024. Accessed 2026-08-24.

- [4] Oven Inc. Bun: All-in-one JavaScript Runtime. <https://bun.sh>, 2024. `bun:ffi dlopen`, backed by JavaScriptCore.
- [5] WebAssembly Working Group. WebAssembly Core Specification, Version 2.0. W3C, 2023. <https://www.w3.org/TR/wasm-core-2/> (W3C Recommendation, June 2023).
- [6] Rust and WebAssembly Working Group. wasm-pack and wasm-bindgen. <https://rustwasm.github.io/docs/wasm-pack/>, 2024. Now at <https://drager.github.io/wasm-pack/> after org sunset July 2025.
- [7] Klabnik, S., Nichols, C. *The Rust Programming Language*, 2nd ed. No Starch Press, 2023. <https://doc.rust-lang.org/book/> (online ed. continuously updated).
- [8] Natural Project. natural: General natural language facility for Node.js. <https://github.com/NaturalNode/natural>, 2024. npm `natural`, MIT.
- [9] Wink JS. winkNLP: High-performance JavaScript NLP. <https://winkjs.org/wink-nlp/>, 2024.
- [10] Kelly, S. compromise: Natural language processing in JavaScript. <https://github.com/spencermountain/compromise>, 2024.
- [11] Honnibal, M., Montani, I. spaCy: Industrial-strength Natural Language Processing. Explosion AI, 2023. <https://spacy.io>
- [12] Cloudflare, Vercel. Cloudflare Workers and Vercel Edge Runtime: WASM-only code on the edge. <https://workers.cloudflare.com/> and <https://vercel.com/docs/functions/edge-functions>, 2024.
- [13] Qi, P., Zhang, Y., Zhang, Y., Bolton, J., Manning, C.D. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In *Proc. ACL System Demonstrations*, pp. 101–108, Online, 2020. Association for Computational Linguistics. <https://aclanthology.org/2020.acl-demos.14/>
- [14] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M. Transformers: State-of-the-Art Natural Language Processing. In *Proc. EMNLP System Demonstrations*, pp. 38–45, Online, 2020. Association for Computational Linguistics. <https://aclanthology.org/2020.emnlp-demos.6/>
- [15] AXA Group, Seijas, J., et al. nlp.js: An NLP library for building bots, with entity extraction, sentiment analysis and language detection. <https://github.com/axa-group/nlp.js>, 2024. npm `nlp.js`, Apache-2.0.
- [16] Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A., Bast, J.F. Bringing the Web up to Speed with WebAssembly. In *Proc. PLDI*, pp. 185–200, 2017. ACM. <https://doi.org/10.1145/3062341.3062363>